# 第20课 选择排序与插入排序

---

## 一、课程信息

| 项目 | 内容 |
| --- | --- |
| 课次 | 第 20 课 / 共 40 课 |
| 主题 | 选择排序与插入排序 |
| 前置知识 | 第 19 课：冒泡排序（相邻比较交换、双重循环结构） |
| 后续衔接 | 第 21 课：sort 函数与 STL 入门 |
| 课时 | 100 分钟 |

### 教学目标（可检验）

1. **能独立默写选择排序的完整代码**，对任意 n 个整数能正确升序排列并输出。
2. **能独立默写插入排序的完整代码**，对任意 n 个整数能正确升序排列并输出。
3. **能用自己的话说出冒泡、选择、插入三种排序的核心区别**，并在给定场景中选择合适的排序方法完成编程。

---

## 二、教学重点与难点

### 重点

- 选择排序的核心思路：每轮从"未排序区"找最小值，放到"已排序区"末尾
- 插入排序的核心思路：依次取"未排序区"第一张牌，插入"已排序区"正确位置
- 三种排序算法（冒泡、选择、插入）的思路对比

### 难点

- 选择排序中"记录最小值下标"而不是记录最小值本身（先记位置，最后再交换）
- 插入排序中"边比较边挪动"的过程（`while` 循环从后往前挪元素）
- 插入排序中 `temp` 变量的作用：为什么要先把牌抽出来保存

---

## 三、课堂时间分配（100 分钟）

| 时间 | 时长 | 环节 | 内容 |
|---|---|---|---|
| 0′–5′ | 5 min | 热身回顾 | 快速回顾冒泡排序的核心思路（师生互动问答） |
| 5′–25′ | 20 min | 讲解① | 选择排序原理（体育课排队故事 + 代码演示） |
| 25′–40′ | 15 min | 练习① | 学生动手：例题 1（选择排序）+ 练习 1 |
| 40′–45′ | 5 min | 休息/趣味互动 | "谁是最矮的"小游戏（口头模拟选择排序） |
| 45′–65′ | 20 min | 讲解② | 插入排序原理（扑克牌整理故事 + 代码演示） |
| 65′–80′ | 15 min | 练习② | 学生动手：例题 2（插入排序）+ 练习 2 |
| 80′–92′ | 12 min | 讲解③ + 综合对比 | 三种排序大比拼（对比表格 + 例题 3） |
| 92′–100′ | 8 min | 总结 + 作业布置 | 知识点回顾、布置洛谷作业、下节预告 |

> **说明**：三段讲解分别为 20 min、20 min、12 min，均不超过 25 分钟。每段讲解后紧跟动手练习或互动环节，保持注意力。

---

## 四、讲稿提纲

### 4.1 热身回顾（5 min）

> 同学们，上节课我们学了第一种排序方法——**冒泡排序**。还记得冒泡排序是怎么干活的吗？
>
> 对！冒泡排序就像水里的气泡一样，**大的泡泡一点一点往上冒**。每一轮，相邻的两个数比一比，大的往后挪，一轮下来最大的数就"冒"到了最后面。
>
> 快速抢答：
> - 冒泡排序需要几层循环？（两层 `for`）
> - 每一轮结束后，哪个位置的数已经确定了？（最后面那个，是当前最大的）
> - 5 个数最多需要几轮？（4 轮）
>
> 很好！今天我们要学**两种全新的排序方法**——**选择排序**和**插入排序**。它们和冒泡排序**目的相同**（把数从小到大排好），但**干活的思路完全不同**！就像去学校有三条路，终点一样，走法不同。

## 4.2 讲解①：选择排序（20 min）

### 故事引入——体育课排队

> 同学们上体育课排队的时候，体育老师是怎么排的？
>
> 想象一下：体育老师让全班 30 个同学站成一排，然后开始排队。老师的方法是——
>
> **第 1 步**：老师用眼睛"扫"一遍所有人，找到**最矮的那个同学**，让他走出来，站到**第 1 个位置**。
> **第 2 步**：老师再"扫"一遍剩下的人，找到**剩下人里最矮的**，站到**第 2 个位置**。
> **第 3 步**：再扫一遍剩下的……
> ……
>
> **一直重复**，直到所有人都站好了！
>
> 这就是**选择排序**的思路：**每一轮，从还没排好的部分里"选"出最小的，放到正确位置**。

### 关键概念

> 我们把数组想象成两个区域：
> - 左边是**"已排好区"**（站好的同学）
> - 右边是**"未排好区"**（还在乱站的同学）
>
> 一开始，所有人都在"未排好区"。每做一轮，就从未排好区挑一个最小的，放到已排好区的最后面。已排好区就多了一个人，未排好区就少了一个人。

### 板书/PPT 演示

用具体数字演示过程：`a[] = {5, 3, 8, 1, 4}`

| 轮次 | 未排好区 | 找到最小值 | 交换操作 | 排完后数组 |
|---|---|---|---|---|
| 第 1 轮 | `[5, 3, 8, 1, 4]` | 1（下标 3） | `a[0]` ↔ `a[3]` | **1**, 3, 8, 5, 4 |
| 第 2 轮 | `[3, 8, 5, 4]` | 3（下标 1） | 已在正确位置 | 1, **3**, 8, 5, 4 |
| 第 3 轮 | `[8, 5, 4]` | 4（下标 4） | `a[2]` ↔ `a[4]` | 1, 3, **4**, 5, 8 |
| 第 4 轮 | `[5, 8]` | 5（下标 3） | 已在正确位置 | 1, 3, 4, **5**, 8 |

> 4 轮结束，5 个数排好了！注意：**5 个数只需要 4 轮**（最后一个不用选，自然就是最大的）。

### 代码讲解要点

> 选择排序有一个很聪明的小技巧：我们不是找到最小的数就搬过来，而是**先记住最小数的"位置"（下标）**，最后再交换。
>
> 为什么呢？就像体育老师不是看到一个矮的就让他往前挤（那会很乱），而是**先用手指着最矮的那个人**，全扫完了再让他走到前面。
>
> 我们用一个变量 `minIdx` 来"指着"当前最小的人的位置。

**易错点强调**

> 注意：是记录**下标**（位置），不是记录值！
> - `minIdx = i;` 记住了"第几号位置"
> - `minVal = a[i];` 只记了值，不知道在哪，没法交换！
>
> 口诀：**"选择排序记下标，找到位置再交换"**。

## 4.3 休息 + 趣味互动（5 min）

> 小游戏时间！请 5 位同学站起来，每人举一个数字卡片。我们用**选择排序的方法**，全班同学一起帮忙找最小的：
> "第 1 轮，大家帮我看看谁最小？……好，找到了，请你走到第 1 个位置！"
>
> 通过身体力行地做一遍，加深记忆。

## 4.4 讲解②：插入排序（20 min）

**故事引入——整理扑克牌**

> 同学们打过扑克牌吗？发牌的时候，你们是怎么整理手牌的？
> 是不是这样：一开始手上没有牌。每抓到一张新牌，你就在手里**从右往左**找一个合适的位置，把新牌**插进去**。比如：
>
> - 手上是空的，抓到 **7**，直接拿着：`[7]`
> - 抓到 **3**，3 比 7 小，插到 7 前面：`[3, 7]`
> - 抓到 **5**，5 比 7 小、比 3 大，插到中间：`[3, 5, 7]`
> - 抓到 **2**，2 比所有都小，插到最前面：`[2, 3, 5, 7]`
>
> 你看，每抓一张牌，手里的牌**始终是排好序的**！这就是**插入排序**的思路。

**对应到数组**

> 在数组里，我们假设 `a[0]` 一开始就是"手里的第一张牌"（一张牌当然是有序的）。
> 然后从 `a[1]` 开始，每个元素都是"新抓到的牌"：
> 1. 先把新牌**抽出来**，存到一个临时变量 `temp` 里（不然会被覆盖！）
> 2. 从这张牌的**前一个位置开始往左看**
> 3. 如果左边的牌**比新牌大**，就把它**往右挪一格**（给新牌腾位置）
> 4. 直到找到一个**不比新牌大**的位置，把新牌**插进去**！

**板书/PPT 演示**

用具体数字演示过程：`a[] = {5, 3, 8, 1, 4}`

**第 1 轮**：新牌 = `a[1]` = 3，`temp = 3`
- 看 `a[0]` = 5，5 > 3，把 5 往右挪 数组变成 `[5, 5, 8, 1, 4]`
- 左边没牌了，把 3 插到位置 0 数组变成 `[3, 5, 8, 1, 4]`

**第 2 轮**：新牌 = `a[2]` = 8，`temp = 8`
- 看 `a[1]` = 5，5 < 8，不用挪！8 就待在原位 `[3, 5, 8, 1, 4]`

**第 3 轮**：新牌 = `a[3]` = 1，`temp = 1`
- 看 `a[2]` = 8，8 > 1，挪 `[3, 5, 8, 8, 4]`
- 看 `a[1]` = 5，5 > 1，挪 `[3, 5, 5, 8, 4]`
- 看 `a[0]` = 3，3 > 1，挪 `[3, 3, 5, 8, 4]`
- 左边没牌了，插到位置 0 `[1, 3, 5, 8, 4]`

**第 4 轮**：新牌 = `a[4]` = 4，`temp = 4`
- 看 `a[3]` = 8，8 > 4，挪 `[1, 3, 5, 8, 8]`
- 看 `a[2]` = 5，5 > 4，挪 `[1, 3, 5, 5, 8]`
- 看 `a[1]` = 3，3 < 4，停！插到位置 2 `[1, 3, 4, 5, 8]`

> 排好了！

**关键理解**

> 插入排序有两个要点：
> 1. **`temp` 变量是"安全口袋"**：把新牌先放进口袋里保护好，不然在挪动的时候它会被覆盖掉！
> 2. **挪动方向是从右往左**：用 `while` 循环，`j` 从当前位置前一个开始往左走，碰到比 `temp` 大的就往右推一格。

## 4.5 讲解③：三种排序大比拼（12 min）

> 现在我们已经学了三种排序方法了！来做一个对比：

| 对比项 | 冒泡排序 | 选择排序 | 插入排序 |
|---|---|---|---|
| 比喻 | 大泡泡往上冒 | 体育老师挑最矮的 | 整理扑克牌 |
| 核心操作 | 相邻两两比较交换 | 找最小值放到前面 | 把新元素插到合适位置 |
| 交换次数 | 多（每次比较都可能交换） | 少（每轮最多交换一次） | 不是交换，是挪动+插入 |
| 特点 | 最好理解 | 交换次数最少 | 对基本有序的数据很快 |
| 时间复杂度 | $O(n^2)$ | $O(n^2)$ | $O(n^2)$ |

> 三种排序，**目的相同，思路不同**！就像去学校可以走大路、小路、或者骑自行车，终点一样，走法不同。
> 有同学可能会问："那到底用哪种好？"答案是——**都可以！** 而且下节课我们还会学一个超级厉害的武器 `sort` 函数，一行代码就搞定排序，比这三种都快！但是，**理解这三种排序的思路非常重要**，因为竞赛题里经常考排序的思想。

---

## 七、常见错误与应对

### 错误 1：选择排序中记录值而不是下标

```
int minVal = a[i];  //  只记了值
for (int j = i + 1; j < n; j++) {
    if (a[j] < minVal) {
        minVal = a[j];  // 记了新的最小值，但不知道它在哪!
    }
}
// 接下来怎么交换？不知道 minVal 对应的下标是几!
```

**正确做法：**

```
int minIdx = i;  //  记住下标（位置）
for (int j = i + 1; j < n; j++) {
    if (a[j] < a[minIdx]) {
        minIdx = j;  // 更新最小值的下标
    }
}
// 现在用 a[i] 和 a[minIdx] 交换就行了
```

**口诀："选择排序记位置，知道在哪才能换"。**

---

### 错误 2：插入排序忘记保存 temp

```
for (int i = 1; i < n; i++) {
    int j = i - 1;
    while (j >= 0 && a[j] > a[i]) {  //  a[i] 已经被覆盖了!
        a[j + 1] = a[j];
        j--;
    }
    a[j + 1] = a[i];  //  a[i] 的值早就变了!
}
```

**问题：**当 `a[j]` 往右挪到 `a[j+1]` 时，如果 `j+1` 刚好是 `i`，那 `a[i]` 原来的值就被覆盖了！

**正确做法：**

```
int temp = a[i];  //  先把新牌放进安全口袋
int j = i - 1;
while (j >= 0 && a[j] > temp) {  //  用 temp 比较
    a[j + 1] = a[j];
    j--;
}
a[j + 1] = temp;  //  用 temp 插入
```

**口诀："新牌先进口袋里，不然挪着挪着就丢了"。**

---

### 错误 3：插入排序 while 条件顺序写反

```
while (a[j] > temp && j >= 0) {  //  危险! j 可能已经是 -1 了
```

**问题**：当 `j = -1` 时，先访问 `a[-1]`，这是数组越界！C++ 不会报错但结果不可预测。

**正确做法**：

```
while (j >= 0 && a[j] > temp) {  //  先检查 j 合法，再访问数组
```

**口诀："先看下标合不合法，再看数值大不大"**（短路求值：`j >= 0` 不满足时，后面的 `a[j]` 根本不会执行）。

---

### 错误 4：选择排序外层循环写成 i < n

```
for (int i = 0; i < n; i++) {  //  多循环了一轮
```

**问题**：最后一轮只剩一个数，不需要再"选"了。多循环一轮虽然结果不会错（自己跟自己比），但浪费时间。

**正确做法**：

```
for (int i = 0; i < n - 1; i++) {  //  n 个数只需要 n-1 轮
```

**口诀："n 个数排 n-1 轮，最后一个自然归位"。**

---

### 错误 5：交换两个数时漏了 temp

```
a[i] = a[minIdx];
a[minIdx] = a[i];  //  此时 a[i] 已经变了，两个变成一样的了！
```

**正确做法：**

```
int temp = a[i];      //  三步交换法
a[i] = a[minIdx];
a[minIdx] = temp;
```

**口诀："交换要靠小助手，三步交换不能少"**（这个上节课冒泡排序也强调过！）。

---

## 九、下节课预告

> 同学们，今天我们学了选择排序和插入排序，加上上节课的冒泡排序，你们已经会**三种排序方法**了！
> 有没有觉得每次写排序都要写好多行代码？有没有想过："要是一行代码就能排好序该多好？"
> 好消息！下节课我们就来学一个**超级强大的魔法武器**——**`sort` 函数**！它是 C++ 标准库（STL）里自带的排序工具，**一行代码搞定排序，而且速度比我们手写的快很多**！
> 到时候你们就会明白，为什么作业 2 会超时——因为我们手写的排序太慢了，而 `sort` 函数用了更聪明的排序方法。
> 下节课还会带大家认识 STL 这个"百宝箱"，里面有很多现成的工具。学会了它们，以后做题就如虎添翼了！
> 回家记得完成作业，下节课见！

---

**备课笔记（仅教师参考）**

- 本课是整个排序单元的第二课，学生已有冒泡排序基础，重点在于**对比三种排序思路的差异**。不要求学生理解时间复杂度的数学推导，但要建立"三种都是 $O(n^2)$，都比较慢"的直觉。
- 选择排序的"记下标"和插入排序的"保存 temp"是本课两个最容易出错的点，务必在演示时用高亮颜色标注，并让学生跟着手写一遍。
- 扑克牌比喻效果很好，建议准备一副大号教学扑克牌，现场演示插入排序过程。
- 如果课堂时间紧张，练习 4（选做挑战）可以留作课后思考题。
- 例题 3 的"三合一"代码较长，建议让学生先自己写选择排序部分，冒泡排序部分直接抄上节课笔记，插入排序部分对照例题 2 补全。不必要求所有学生在课堂上全部完成。
- 作业 2（P1177 模板排序）故意让学生体验 TLE，是为下节课引入 `sort` 做心理铺垫。如果有学生因为 TLE 而沮丧，请安慰他们：**这不是你们写错了，而是方法不够快，下节课就有解决办法了！**
- 三种排序对比表建议打印出来或做成海报贴在教室里，方便学生随时查阅。